Python Notes: The for Loop

Lesson Introduction
In this lesson, we learn about the **for loop** in Python. A loop allows a program to repeat a block of code multiple times. Previously, we discussed loops that depend on conditions (such as while loops). In contrast, a for loop is designed to iterate directly over a sequence of values.

A for loop is commonly used when:

- You want to go through every element in a list.
- You want to examine each character in a string.
- You want to generate and iterate through a sequence of numbers.

The for loop simplifies iteration because it does not require manually updating a counter variable.

1. Difference Between while and for Loops

while Loop

- Runs **as long as a condition is True**.
- Requires manual control of a variable.
- You must update the loop variable yourself.
- Risk of infinite loops if the condition never becomes False.

for Loop

- Iterates directly over a sequence (list, string, tuple, etc.).
- No need to manually increment or decrement a variable.
- Cleaner and less error-prone for sequence-based iteration.

2. Basic for Loop Syntax

Code

```
for variable in sequence:
    indented block of code
```

Explanation:

- `variable` → Temporary variable that takes each value from the sequence.
- `sequence` → Any iterable object (list, tuple, string, or range).
- The indented block defines what happens each iteration.

Indentation is required in Python. Without correct indentation, the loop will not work properly.

3. Iterating Over a List

Example:

Python

```python
my_list = ["Naveen", 65, 2.5]

for item in my_list:
    print(item)
```

What happens step-by-step:

1. `item` becomes `"Naveen"` → printed.
2. `item` becomes `65` → printed.
3. `item` becomes `2.5` → printed.

Each element is accessed one at a time automatically.

4. Iterating Over a String

Strings are sequences of characters.

Example:

Python

```python
name = "Naveen"

for ch in name:
    print(ch)
```

What happens:

- `ch` becomes `'N'`
- then `'a'`
- then `'v'`
- then `'e'`
- then `'e'`
- then `'n'`

Each character is processed individually.

5. The range() Function

The `range()` function generates a sequence of numbers. It is commonly used with `for` loops.

Basic Form

Python

```python
range(stop)
```

Example:

Python

```python
for i in range(5):
    print(i)
```

Output:

Code

```
0
1
2
3
4
```

Important: The stop value is **exclusive** (not included).

Two Parameters

Python

```python
range(start, stop)
```

Example:

Python

```python
for i in range(11, 21):
    print(i)
```

Output:

Code

```
11 12 13 14 15 16 17 18 19 20
```

Three Parameters

```python
range(start, stop, step)
```

Example:

```python
for i in range(11, 21, 2):
    print(i)
```

Output:

```
11 13 15 17 19
```

The `step` determines how much the number increases each time.

6. Reverse Iteration

To loop backwards, use a negative step.

Example:

```python
for i in range(20, 10, -1):
    print(i)
```

Output:

```
20 19 18 17 16 15 14 13 12 11
```

Notice:

- Start = 20
- Stop = 10 (exclusive)
- Step = -1 (moves backward)

7. Using if Statements Inside a for Loop

You can combine loops with conditionals to control behavior.

Example: Print numbers from 1 to 20, skipping multiples of 5.

Python

```python
for i in range(1, 21):
    if i % 5 != 0:
        print(i)
```

Explanation:

- `%` is the modulo operator.
- `i % 5 == 0` means the number is divisible by 5.
- The condition filters out those numbers.

This allows selective processing inside loops.

8. Important Rules

- Indentation defines the loop block.
- The loop variable automatically changes each iteration.
- range() stop value is always exclusive.
- Negative step enables descending loops.
- Conditionals inside loops allow filtering.

Summary Table

Concept	Description	Example
`for` loop	Iterates over a sequence	`for i in [1,2,3]:`
List iteration	Access each list element	`for x in my_list:`
String iteration	Access each character	`for ch in "Hello":`
Tuple iteration	Works like list iteration	`for n in (1,2,3):`
`range(stop)`	0 to stop-1	`range(5)`
`range(start, stop)`	Start to stop-1	`range(10,15)`
`range(start, stop, step)`	Custom increment	`range(1,10,2)`
Reverse loop	Negative step	`range(10,0,-1)`
Conditional in loop	Filter values	`if i % 5 != 0:`